AI-Driven Framework for End-to-End Software Testing Lifecycle Automation

Jinad Induvidwa A.G., Abeygunasekara D. T., Ranasinghe S. I., Rashani K.G.M., Suriyaa Kumari, Eishan Weerasinghe
Faculty of Computing
Sri Lanka Institute of Information Technology, Malabe, Sri Lanka
{it22251114@my.sliit.lk, it22303684@my.sliit.lk, it22371768@my.sliit.lk}
{it22182678@my.sliit.lk, suriyaa.k@sliit.lk, eishan.w@sliit.lk}

***Abstract*—Software quality assurance spans requirement elicitation, test design, execution, defect analysis, traceability, and coverage assessment, yet these activities are often supported by disconnected tools and manual handoffs. This research proposes an AI-driven, human-governed framework for end-to-end Software Testing Lifecycle automation: live meeting conversations are transcribed and analyzed to identify and validate requirements, resolve conflicts, and generate evidence-linked user stories, which become executable test scenarios and automated test suites across multiple platforms; execution failures are analyzed for root causes with controlled, developer-approved self-healing; and traceability and code coverage are continuously maintained from executed test evidence, with human approval gates keeping AI-generated artifacts subject to explicit validation. The approach is evaluated on requirement processing, test executability, fault detection, self-healing effectiveness, traceability correctness, and code coverage, aiming to reduce manual effort and provide integrated, intelligent lifecycle automation.**

***Keywords*—software testing lifecycle, requirements traceability, large language model agents, automated test generation, self-healing software**

I. Introduction

Modern Agile software delivery relies on many human and tool-mediated handoffs across the software testing lifecycle. Requirements are often discussed verbally in sprint meetings and later transformed into backlog items, inviting context loss, misinterpretation, and unresolved conflicts; converting them into executable tests is time-consuming and expertise-dependent. Although large language models (LLMs) accelerate test generation, generated tests may contain invalid element references, omit acceptance criteria, or fail to execute. CI/CD failures often require manual diagnosis, unrestricted automated repair is risky when AI agents may modify repositories, and traceability and coverage information are frequently maintained apart from actual execution, reducing timeliness and reliability.

Recent research demonstrates AI's potential for individual testing activities – requirements analysis, automated test generation, failure analysis, and program repair – but these capabilities are commonly separate solutions rather than a unified workflow connecting requirements to executable tests, execution evidence, defect analysis, repair, traceability, and coverage, leaving a gap in end-to-end, evidence-grounded automation. Fully autonomous decision-making also remains challenging, since AI-generated artifacts and code modifications may require human validation before acceptance.

To address this gap, this research proposes an AI-driven framework integrating conversational requirement elicitation and validation, automated test generation, multi-framework execution, failure analysis, controlled self-healing, and execution-based traceability and coverage: requirements captured from live meetings become validated user stories and then executable test scenarios, execution results drive failure analysis and controlled repair, and all artifacts remain linked through a shared traceability layer. A key design principle is human governance: AI components analyze, generate, and propose, but consequential artifacts must pass explicit approval gates, and repository modifications are restricted by a deterministic safety policy with developer approval.

The research investigates four questions. **RQ1:** Can requirement-bearing utterances be reliably extracted, threaded, and conflict-checked from live meeting conversations to produce validated and traceable user stories? **RQ2:** Can grounded, coverage-driven agentic test generation produce executable test suites with improved executability and coverage over baseline approaches? **RQ3:** Can CI/CD test failures be accurately classified and safely handled through deterministic self-healing while preventing unauthorized repository modifications? **RQ4:** Can test quality, requirements traceability, gap analysis, and code coverage be maintained using evidence from actual test execution?

Addressing these questions, the main contributions are context-aware requirement harmonization with human validation (RQ1), coverage-driven grounded agentic test generation (RQ2), safety-governed self-healing with deterministic rules and developer approval (RQ3), grounded quality, traceability, and coverage measurement (RQ4), and their integration into one end-to-end, human-governed testing framework evaluated on requirement processing, executability, fault detection, self-healing effectiveness, safety compliance, traceability, and coverage.

II. Literature Review

A. Theoretical and Technical Foundations

Requirements engineering and traceability. Requirements traceability maintains relationships between requirements and downstream artifacts [1]; Agile methodologies represent functional requirements as user stories [2], with the INVEST principles as story-quality criteria [3]. However, these approaches generally assume already-documented requirements, whereas in real Agile environments requirements are negotiated through meetings where information is incomplete, contextual, and subject to change – creating a need to capture conversational requirements, preserve context, identify conflicts, and transform them into validated, traceable artifacts.

LLMs for software testing. LLMs can generate unit tests from source code and natural-language descriptions [10], and CodaMosa uses pre-trained models to escape coverage plateaus in automated test generation [11]. Retrieval-augmented generation (RAG) improves grounding by retrieving relevant external or application-specific information before generation [4]. Nevertheless, generated tests may still contain incorrect selectors, incomplete assertions, or scenarios that misrepresent the intended requirements, motivating application-specific grounding, explicit validation, and execution-based verification.

Agentic software exploration and visual grounding. ReAct combines reasoning and action for interactive agent behavior [6], WebArena provides a realistic evaluation environment for autonomous web agents [9], Set-of-Mark prompting improves visual grounding over explicitly identified interface elements [7], and Reflexion adds verbal feedback and memory to improve subsequent behavior [8]. These approaches demonstrate strong autonomous interaction, but their primary objective is task completion rather than systematic test coverage, so a testing-oriented exploration strategy requires explicit state tracking, failure memory, and coverage-based termination criteria.

CI/CD failure analysis and automated program repair. CI environments generate failures from heterogeneous sources – tests, dependencies, configuration, infrastructure, and application defects – and build failures arise from multiple recurring causes rather than one category [13]. LLM-based repair agents show repository-level automated repair is increasingly feasible [14], while SWE-bench illustrates the difficulty of resolving realistic GitHub issues automatically [15]. However, increasing repair autonomy introduces security and reliability concerns when agents hold repository write access, so practical self-healing requires explicit authorization, restricted permissions, and safety policies separating failure prediction from permission to modify source code.

Test quality, prioritization, and coverage. Mutation testing estimates the fault-detection capability of test suites [16], risk-based testing prioritizes activities by failure impact and likelihood [17], and prioritization measures such as APFD quantify how quickly faults are detected during regression testing [12]. These techniques provide strong foundations for evaluating AI-generated tests, yet quality scores, coverage measurements, and traceability are often separate artifacts; an integrated lifecycle requires them to remain continuously connected to requirements, tests, execution results, and approval states.

B. Comparative Positioning and Research Gap

Table I positions the proposed framework against representative approaches across the major lifecycle dimensions.

TABLE I
Comparative Positioning of the Proposed Framework

Approach	Live Req.	Req. Val.	LLM Test	Multi-FW	Agentic	Repair	RTM/Cov.
Traditional RE	×	✓	×	×	×	×	Partial
LLM Test Gen.	×	Partial	✓	Partial	×	×	Partial
CodaMosa	×	×	✓	×	Partial	×	✓
ReAct / Web Agents	×	×	Partial	×	✓	×	Partial
Repair Agents	×	×	×	×	×	✓	Partial
Traditional RTM	×	Partial	×	Partial	×	×	✓
Proposed	✓	✓	✓	✓	✓	✓	✓

Note: ✓ = supported; Partial = partial support; × = not supported. RE = Requirements Engineering; FW = Frameworks; RTM = Requirements Traceability Matrix; Cov. = Execution Coverage.

In summary, existing approaches operate on documented rather than live conversational requirements, generate tests that are insufficiently grounded in the target application, explore applications for task completion rather than systematic coverage, grant repair agents autonomy without sufficient safety governance, and maintain traceability, quality, and coverage as separate artifacts detached from execution evidence. The proposed framework addresses this gap by integrating the lifecycle as a continuous workflow: *requirement elicitation → story validation → test generation → agentic exploration → execution → failure analysis → controlled self-healing → quality, traceability, and coverage*.

III. Methodology

A. Architecture and Pipeline Integration

The framework comprises five independently deployable microservices – Authentication, Intelligent User Story Generation, Test Case Generation, Self-Healing, and Requirements Traceability Matrix (RTM) Generation – forming a continuous pipeline through defined APIs. The Authentication Service manages authentication and authorization; the remaining four form a sequential pipeline in which validated user stories flow into test generation, execution failures into self-healing, and requirement, test, execution, and coverage information into the RTM.

Each service follows the database-per-service principle: services never access each other's databases directly and communicate only through defined APIs and service-to-service interfaces.

B. Intelligent User Story Generation Service

The Intelligent User Story Generation Service (Fig. 1) transforms live multi-speaker meeting conversations into validated Agile user stories through real-time speech processing, contextual requirement classification, requirement threading, conflict detection, RAG-based story generation, and multi-layer validation.

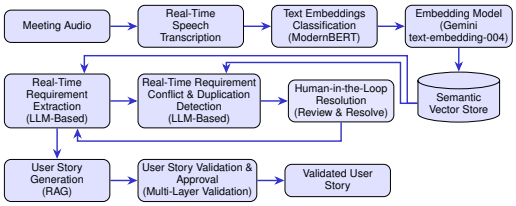


Fig. 1. Intelligent User Story Generation workflow.

1) Real-Time Requirement Processing: Meeting audio is streamed over WebSocket and processed with the Azure Cognitive Services Speech SDK for real-time speech-to-text and speaker diarization; each utterance is embedded with Gemini `text-embedding-004` and stored in the service-specific PostgreSQL database with `pgvector`. A finetuned ModernBERT [5] classifier categorizes each utterance with the previous, current, and next utterances as context into nine categories – *Requirement, Suggestion, Decision, Action Item, Risk, Clarification, Question, Rejected*, and *Future Feature*; Requirement and Suggestion utterances continue for processing.

2) Requirement Threading and Conflict Detection: An LLM extracts structured requirements from relevant utterances, and a Requirement Thread Manager groups semantically related requirements into threads following the lifecycle DISCOVERED → DISCUSSION → REFINED → VALIDATED. For each new requirement, Top-K similar requirements are retrieved via `pgvector` and analysed for five conflict types – functional contradiction, scope overlap, priority conflict, temporal conflict, and resource conflict – with detected conflicts presented to a Business Analyst for resolution, keeping consequential decisions human-controlled.

3) RAG-Based Story Generation and Validation: Validated requirements feed a Retrieval-Augmented Generation (RAG) pipeline: relevant meeting evidence and requirement context are retrieved and given to the LLM to generate user stories and acceptance criteria. Each story passes four validation stages – rule validation, evidence validation, hallucination detection, and INVEST evaluation – each normalized to a 0–100 scale and combined into a single quality score Q as

$$Q = 0.20R + 0.25E + 0.20H + 0.35I \tag{1}$$

where R, E, H, and I denote the rule-validation, evidence-validation, hallucination-detection, and INVEST validation scores, respectively. Stories scoring $\geq$ 80 are APPROVED, 50–79 NEEDS REVIEW, and below 50 REJECTED; approved stories pass to the Test Case Generation Service through the defined interface.

C. Executable Test Generation and Agentic Exploration

The Test Case Generation Service closes the gap between an approved user story and an executed, reported test suite without manual scripting, addressing two practical problems: purely LLM-generated UI tests often fail on first execution because the model has never observed the target application and hallucinates selectors, and every scenario must first be

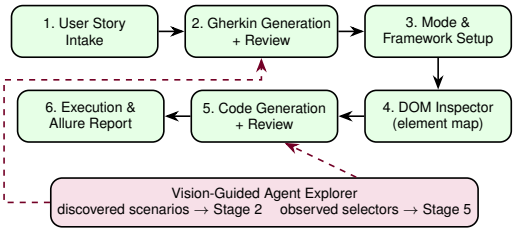


Fig. 2. Six-stage test generation pipeline with Agent Explorer feedback.

imagined by an engineer, so forgotten scenarios are never tested. It responds with application-grounded generation and an autonomous, vision-guided exploration agent, keeping a human reviewer in the loop at every stage: the AI drafts, the engineer approves.

1) Six-Stage Generation Pipeline: Approved user stories flow through six stages (Fig. 2). Stage 1 organizes stories into projects from the Intelligent User Story Generation Service or a structured Actor–Action–Goal intake form. Stage 2 converts each story into Gherkin (Given–When–Then) via curated Jinja2 templates with a pluggable LLM fallback in a Monaco-based editor for edit, regenerate, or approve; only approved scenarios continue – the human-approval invariant relied on by the rest of the pipeline. Stage 3 sets the generation mode, target framework (Selenium, Playwright, or Cypress), and target URL, probed for reachability. Stage 4 runs a Playwright-driven DOM inspector in a real browser, authenticating if required and capturing every interactive element into an engineer-approved element map; a deterministic selector builder prefers `id`, `data-testid`, `name`, then `aria-label`, yielding stable, auditable selectors. Stage 5 combines approved Gherkin, element map, and framework to emit a complete suite per framework in tabbed editors for review, and Stage 6 dispatches it to a clean GitHub Actions runner with live logs, screenshots, and Allure reporting; outcomes are persisted for the Self-Healing and traceability services.

2) Grounded Test Generation as a Built-In Ablation: Stage 5 generates the same approved Gherkin in two modes: Mode A (abstract) needs no application access and emits locators as named placeholders such as `<USERNAME_FIELD>`—fast, offline, useful for early planning, but not executable out of the box—while Mode B (DOM-grounded) generates against the approved element map and yields ready-to-run code. Both modes consume identical stories through an identical execution pipeline, so their executability difference is a controlled measurement of grounding value (M1 in Table II).

3) Vision-Guided Agentic Exploration: The pipeline automates test *writing*; the Agent Explorer removes the remaining bottleneck of imagining scenarios. Given one intent sentence (e.g., "check the login flow") and a URL, an autonomous agent opens a real Chromium browser, perceives the page visually, decomposes the intent into sub-goals, acts, observes, learns from failures, and returns validated Given–When–Then scenarios—often including behaviors never requested, such as a locked-out-user flow discovered during login exploration.

In each iteration (Fig. 3), a JavaScript injector draws a numbered overlay on every visible interactable element—

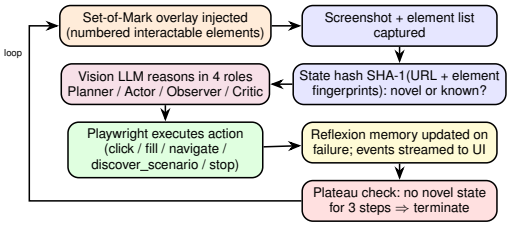


Fig. 3. One agent iteration; the cycle repeats until a coverage plateau, stop action, or step budget.

the Set-of-Mark technique [7]—without mutating the page's DOM; a screenshot and numbered element list are captured, and a state hash marks novel UI states, computed as

$$h = \text{SHA-1}\big(\text{URL} \,\|\, \text{sort}(E)\big)$$

where E is the set of extracted element fingerprints on the current page and $\|$ denotes string concatenation. The screenshot, element list, intent, action history, and accumulated lessons are sent in a single call to a vision LLM, which returns four role-tagged thoughts—Planner, Actor, Observer, Critic, in ReAct style [6]—plus one concrete action that Playwright executes; one call instead of four cuts latency and cost roughly fourfold while sharing context. Choosing numbered marks rather than free-form CSS eliminates selector hallucination by construction, since the agent can only pick "element 7," whose selector was extracted deterministically from the real page. On failure, the agent writes a short lesson injected into every subsequent prompt—the Reflexion pattern [8]—so mistakes are not repeated within a run.

4) Coverage-Driven Termination: Instead of a fixed step budget—which wastes budget on exhausted applications or stops early while novel behavior remains—the Agent Explorer treats the *rate of novel-state discovery* as its stopping signal: no novel state hash for three consecutive steps means exploration has saturated and terminates, with an explicit stop action and hard step budget as safety rails. The distinct state hashes double as the reported coverage metric, the plateau parameter is a principled exploration-versus-cost knob, and newly discovered behaviors become Gherkin scenarios subject to the same human approval as story-derived ones.

5) Evaluation Metrics for Test Generation: The component is evaluated by thirteen metrics (Table II) in four families – generation quality (M1–M4), fault detection on seeded defects (M5–M6), agent exploration and ablations (M7–M11), and human assessment (M12–M13) – with every LLM-dependent value reported as mean ± standard deviation over at least five runs.

D. CI/CD Failure Analysis and Controlled Self-Healing

The Failure Analysis and Self-Healing Service analyses failed tests and CI/CD executions, identifies their most likely root causes, and selects an appropriate recovery action, combining a nine-class machine-learning root-cause classifier, a deterministic recovery policy, and a developer-approved controlled repair mechanism (Fig. 4).

TABLE II
TEST GENERATION AND AGENTIC EXPLORATION EVALUATION METRICS

ID	Metric	Target	Evaluation Basis
M1	Executability Rate	$\geq 85\%$	Syntax/locator error-free test execution
M2	Selector Accuracy	$\geq 90\%$	Match with ground-truth UI element map
M3	Generation Latency	< 10 s/scenario	Story-to-Gherkin generation time
M4	Scenario Coverage	$\geq 80\%$	Acceptance criteria mapped to scenarios
M5	Fault Detection Rate	$\geq 70\%$	Seeded defects detected by generated tests
M6	Mutation Score	$\geq 60\%$	Mutants killed by generated test suite
M7	UI-State Coverage	$\geq 1.2\times$	States reached vs. BFS baseline
M8	Agent Efficiency	≤ 25 steps	Steps, tokens, and cost per scenario
M9	Reflexion Ablation	$\geq +10$ pp	Goal completion with/without memory
M10	Scenario Yield	≥ 3/intent	Valid scenarios discovered per intent
M11	Termination Efficiency	$\geq 80\%$	Runs ending at coverage plateau
M12	Gherkin Quality	$\geq 4/5$	QA ratings for readability and correctness
M13	Effort Savings	$\geq 50\%$	Manual writing vs. review/approval time

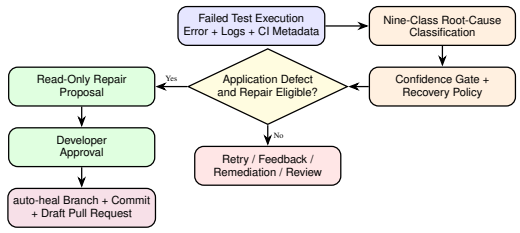


Fig. 4. Failure analysis and controlled self-healing workflow.

1) Failure Evidence and Root-Cause Classification: Failed execution evidence is obtained from the Test Case Generation Service through project-scoped APIs rather than direct database access: the failed test or scenario name, error message, stack trace, execution logs, suite and execution identifiers, and GitHub Actions run metadata when available. The active `organization_id` and `project_id` are retained with each failure, along with upstream iteration, user-story, and test-run identifiers when provided.

A persisted supervised model predicts one of nine root-cause categories – application defect, test-script issue, dependency issue, workflow/environment issue, network issue, infrastructure/resource issue, deployment issue, security-policy issue, and other or unknown – from execution context, error type and message, stack trace, failed source location, and relevant log evidence. This classification is necessary because a failed CI/CD execution may originate from several causes other than an application-code defect [13].

Sensitive evidence is sanitized and bounded before use by the repair workflow; repair attempts retain only the required failure context plus an evidence hash, limiting exposure of raw execution information while preserving auditability.

2) Root-Cause-Based Recovery and Repair Eligibility: The predicted root cause is mapped to a recovery action by a deterministic policy (Table III); only an `application_defect` can enter the controlled self-healing workflow, while other failures receive cause-appropriate actions.

A minimum confidence of 60% is required before controlled repair – lower predictions go to manual review – and eligibility requires verified CI execution metadata, a repository matching the active project configuration, a valid failed commit SHA and source branch, and a valid candidate file and line number, with protected files, absolute paths, and path-traversal locations rejected. The classifier identifies the likely failure category; the deterministic policy independently determines the recovery action.

TABLE III
ROOT-CAUSE-BASED RECOVERY POLICY

Root Cause	Recovery Action
Application defect	Controlled self-healing
Test-script issue	Test-generation feedback / notification
Dependency issue	Dependency remediation
Workflow/environment issue	Workflow remediation
Network issue	Retry pipeline
Infrastructure/resource issue	Retry or resource review
Deployment issue	Rollback or manual review
Security-policy issue	Block and security review
Other or unknown	Manual review

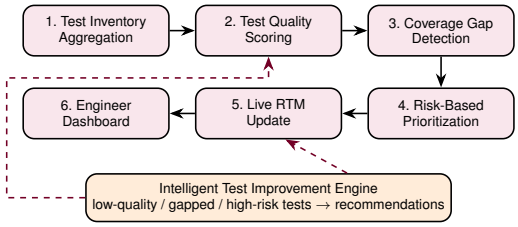


Fig. 5. Quality, coverage, and traceability pipeline with recommendation feedback.

3) Developer-Approved Two-Phase Repair: Eligible application defects follow a two-phase controlled repair. In the read-only first phase, the repair agent receives bounded failure evidence – repository, failed revision, candidate file and location – analyses the repository content, and generates a proposal with the change, rationale, risks, and validation commands; no modification, branch, commit, or pull request is created. The second phase begins only after developer approval: the system revalidates the proposal, repository, failed revision, branch, changed files, protected-path restrictions, and secret-safety conditions, confirming the failed branch still matches the analyzed revision. The repair is applied to an isolated auto-heal/ branch as a single commit with a draft pull request for review; the workflow never modifies the main or base branch and never auto-merges. Audit records retain the predicted root cause, confidence, recommendation, execution traceability, analyzed revision, candidate location, sanitized evidence, proposal, resulting branch, commit, pull-request information, and final outcome.

E. Quality Assessment, Test Inventory, and Live Requirements Traceability

The RTM Generation Service closes the lifecycle by converting executed test evidence into governed decision support: a continuously updated source of truth linking every requirement to its acceptance criteria, generated tests, execution outcomes, and coverage evidence, with three analytical layers – test-case quality scoring, coverage-gap detection, and risk-based prioritization. Consuming execution events from the Test Case Generation Service and repair outcomes from the Self-Healing Service directly, every figure shown reflects the most recently executed repository state rather than a static spreadsheet (Fig. 5).

1) Test Inventory and Unified Test Catalog: Every generated and executed test case is registered in a project-scoped Test Inventory linking it to its originating Gherkin scenario, approved user story and acceptance criterion, target framework, version, latest execution outcome, and current quality and coverage-contribution scores – a single searchable catalog of what exists, what it verifies, and how healthy it is, serving as the join layer for the quality, coverage-gap, risk, and traceability stages.

2) Test Case Quality Assessment: Test quality is evaluated on five weighted characteristics – assertion strength (25%), code-coverage contribution (30%), boundary-value coverage (20%), error-handling coverage (15%), and mutation resistance (10%) [16] – into one interpretable score, cross-checked against a RandomForest model trained on formula-labeled synthetic data [18]. Tests scoring at least 60 are approval-eligible; formula–model divergence beyond a fixed margin is flagged for manual review.

3) Coverage Gap Detection: Coverage values come from repository execution rather than manual assertions: an access-controlled GitHub agent verifies permissions, detects project technology, and runs the appropriate tool (e.g., `nyc`/Istanbul, `coverage.py`). Coverage evidence is mapped to acceptance criteria and tests, written to the RTM only after successful execution; criteria whose linked tests miss the threshold are surfaced as coverage gaps.

4) Risk-Based Prioritization: Coverage gaps and existing tests are ranked by an explainable, keyword-based risk classifier [17]: requirement priority, business-critical keywords (e.g., payment, authentication, checkout), historical defect frequency, and current coverage combine into a composite score bucketed as CRITICAL, HIGH, MEDIUM, or LOW, with each signal explicit and inspectable.

5) Live Requirements-to-Test-Case Traceability: The RTM updates from test approval, execution, and coverage events as they occur, tracking requirement, acceptance criterion, generated test, execution result, and coverage contribution. A requirement is FULLY COVERED when all criteria have an approved, passing test and $\geq$80% average coverage, PARTIALLY COVERED when only some meet this bar, and NOT COVERED otherwise; shared execution events keep the RTM, inventory, and coverage view consistent.

6) Intelligent Test Improvement and Recommendations: Tests scoring poorly, sitting behind a coverage gap, or falling in a CRITICAL/HIGH risk bucket are passed to an LLM-based Recommendation Engine, which inspects assertions, coverage, and the linked requirement to propose improvements such as missing boundary-value assertions or error-handling paths. It returns a reviewable recommendation rather than modifying tests directly; approved recommendations route back to the Test Case Generation Service as regeneration requests, closing the loop under the human-approval invariant.

F. Experimental Setup and Evaluation Metrics

Evaluation spans component and end-to-end levels: labelled meetings (requirement harmonization); SauceDemo (test generation, agentic exploration, selector validation, seeded-fault detection, APFD; random/BFS baselines); 180 balanced CI failure runs across nine root-causes (self-healing);

Python/JavaScript repos (coverage, RTM). Three ablations: grounded vs. ungrounded generation, Reflexion memory on/off, formula-based vs. learned scoring.

Metrics: requirement harmonization (classification macro-F1, conflict-detection precision/recall, validation-score agreement, practitioner assessment); test generation (Table II, APFD); self-healing (root-cause macro-F1, unsafe-modification rate, safety-policy compliance, approval-gate enforcement, duplicate-repair prevention, sensitive-data exposure); quality/RTM (scoring-method agreement, RTM correctness, update latency, coverage-agent success, access-control enforcement, coverage-gap-risk expert agreement).

IV. Results and Discussion

Following the setup and metrics of Section III-F, requirement processing achieved a macro-F1 of **0.94** for nine-class utterance classification, conflict-detection precision and recall of **0.91/0.88**, and an average practitioner story rating of **4.3**/5, indicating effective contextual requirement processing and evidence-grounded story generation.

Grounded test generation achieved **91.2**% executability and **93.5**% selector accuracy, versus **78.6**% and **81.4**% ungrounded, with **87.3**% acceptance-criteria coverage. The agentic explorer reached **84** unique UI states and an APFD of **0.82**, compared with **61/0.68** for random exploration and **76/0.75** for BFS, confirming the value of application grounding and coverage-driven exploration.

The root-cause classifier achieved a macro-F1 of **0.89** on the 180-run dataset, and the controlled repair mechanism achieved **100**% safety-policy compliance and approval-gate enforcement with a **0**% unsafe-modification rate. The formula-based and learned quality-scoring methods correlated at **0.91** (Pearson), RTM status correctness reached **98.7**% with **143** ms update latency, and the coverage agent achieved **94.1**% end-to-end success. Overall, conversational requirements can be connected to executable tests with execution-based traceability, and AI-assisted repair can be governed by deterministic safety policies and developer approval without uncontrolled repository modification – evidence towards the research questions on capability, effectiveness, and safety.

V. Conclusion

NextGen QA closes the quality-assurance loop from a spoken requirement to a verified, traceable, quality-scored test suite—and, when pipelines break, to a safely governed repair. Its contribution is an architecture: classification before generation, evidence before approval, policy before permission, and execution before belief, with capability metrics paired against hard safety invariants. Future work extends the platform to multilingual meetings, additional CI providers, adaptive but policy-subordinated repair strategies, and longitudinal studies of practitioner effort in live teams.

References

[1] O. C. Z. Gotel and C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proc. IEEE Int. Conf. Requirements Engineering*, 1994, pp. 94–101.

[2] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, USA: Addison-Wesley, 2004.

[3] B. Wake, "INVEST in good stories, and SMART tasks," 2003. [Online]. Available: https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] B. Warner et al., "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," *arXiv preprint arXiv:2412.13663*, 2024.

[6] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[7] J. Yang et al., "Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V," *arXiv preprint arXiv:2310.11441*, 2023.

[8] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[9] S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[10] M. Schäfer et al., "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.

[11] C. Lemieux et al., "CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023, pp. 919–931.

[12] G. Rothermel et al., "Prioritizing test cases for regression testing," *IEEE Trans. Software Engineering*, vol. 27, no. 10, pp. 929–948, 2001.

[13] T. Rausch et al., "An empirical analysis of build failures in the continuous integration workflows of Java-based open-source software," in *Proc. IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 345–355.

[14] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," *arXiv preprint arXiv:2403.17134*, 2024.

[15] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[16] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[17] S. Amland, "Risk-based testing: Risk analysis fundamentals and metrics for software testing including a financial application case study," *J. Systems and Software*, vol. 53, no. 3, pp. 287–295, 2000.

[18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.